Banco de Dados

Cláusulas Avançadas, Integridade Referencial e Paginação

Professor: Me. Sergio Souza Novak
Faculdade de Engenharia de Software



Configuração: Google Colab

Nota Importante

Ao utilizar o Google Colab com SQLite, é necessário habilitar manualmente o suporte a chaves estrangeiras executando a seguinte cláusula:

Habilitar Foreign Keys

```
%sql PRAGMA foreign_keys = ON;
```

Sumário

1. Integridade Referencial
2. Cláusulas Avançadas de Consulta
3. Paginação de Dados (LIMIT e OFFSET)
4. Estudo de Caso: Sensores de Solo
5. Conclusão e Prática

O que é Integridade Referencial? (Teoria)

Definição

É um conjunto de regras que garante que a relação entre registros de tabelas diferentes permaneça válida e consistente. No modelo relacional, isso significa que uma referência de uma tabela para outra deve sempre apontar para um dado existente.

- **Consistência**: Garante que não existam referências a dados inexistentes.
- **Confiabilidade**: O banco de dados impede ações que quebrem esses vínculos.
- **Automação**: O SGDB cuida da fiscalização, poupando o trabalho da aplicação.

O Papel da Chave Estrangeira (Teoria)

- A **Chave Estrangeira (Foreign Key)** é o instrumento técnico da integridade referencial.
- Ela é uma coluna (ou conjunto de colunas) em uma tabela que armazena a **Chave Primária** de outra tabela.



Regra de Ouro

O valor na FK deve existir na PK da tabela referenciada ou ser nulo (se a regra do negócio permitir).

Restrições no Modelo Relacional (Teoria)

O Papel das Constraints

As restrições de integridade referencial funcionam como "guardiões" do banco de dados, impedindo que qualquer operação (Manual ou via Aplicação) deixe os dados em um estado inconsistente.

- Elas são verificadas **instantaneamente** pelo SGDB no momento da execução do comando.
- Se a regra for violada, a transação é abortada e o erro é retornado.
- Garante que a "verdade" dos dados seja mantida sem depender da lógica do programador.

Restrição de Inserção (Prática)

Não é permitido criar um registro "filho" que aponte para um "pai" inexistente.



Clientes(id, nome) **Pedidos**(id, id_cliente, valor)
id_cliente referencia Clientes

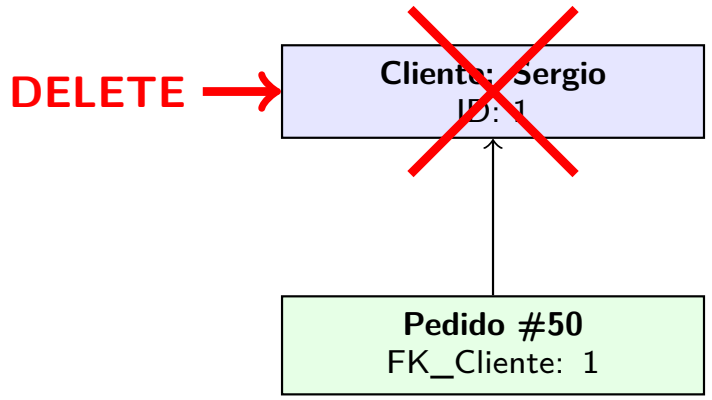
Tentativa de SQL

```
-- O Cliente 999 não existe na tabela Clientes
INSERT INTO Pedidos (id, valor, id_cliente)
VALUES (101, 250.00, 999);
```

Erro: insert or update on table "pedidos" violates foreign key constraint...

Restrição de Exclusão (Prática)

Não é permitido excluir um "pai" enquanto ele possuir "filhos" (sem tratamento de cascata).



Clientes(<u>id</u>, nome) **Pedidos**(<u>id</u>, id_cliente, valor)

id_cliente referencia Clientes

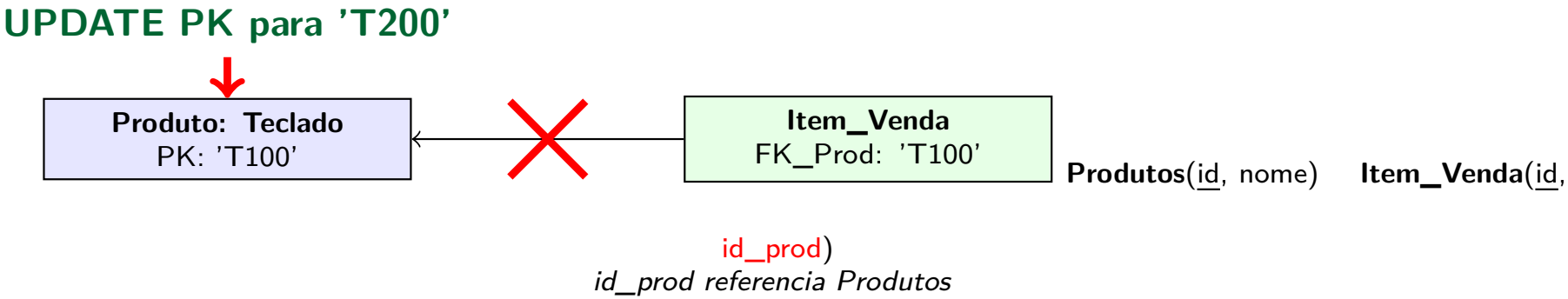
Tentativa de SQL

```
-- Tentar deletar um cliente que já tem compras
DELETE FROM Clientes WHERE id = 1;
```

Erro: update or delete on table "clientes" violates foreign key constraint...

Restrição de Alteração (Prática)

Alterar a Chave Primária do pai "quebraria" o vínculo com os filhos já existentes.



Tentativa de SQL

```
-- Tentar mudar o código do produto que já foi vendido
UPDATE Produtos SET id = 'T200' WHERE id = 'T100';
```

Erro: update or delete on table "produtos" violates foreign key constraint...

Consistência de Domínios e Tipos (Teoria)

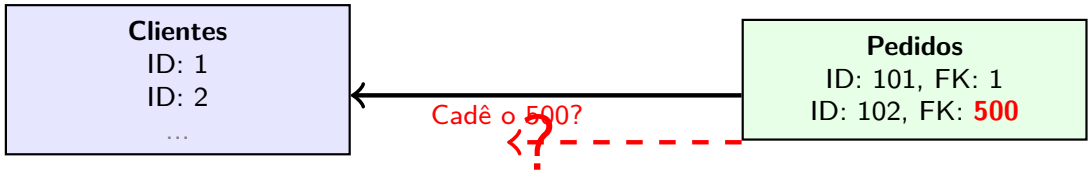
- Para que a integridade funcione, o **Domínio** (tipo de dado) da FK deve ser idêntico ao da PK.
- Se a PK é `INT`, a FK deve ser `INT`.
- Se a PK é `UUID`, a FK deve ser `UUID`.

Por que isso importa?

Tipos incompatíveis impedem que o SGDB realize as verificações de igualdade necessárias para validar o vínculo, além de causarem erros graves de performance em *JOINs*.

Problema 1: Registros Órfãos (Integridade)

Um registro órfão ocorre quando um "filho" referencia um "pai" que não existe mais.



Clientes(id, nome) **Pedidos**(id, id_cliente, valor)

id_cliente referencia Clientes

Como detectar órfãos (SQL)

```
SELECT p.id, p.id_cliente FROM Pedidos p
LEFT JOIN Clientes c ON p.id_cliente = c.id
WHERE c.id IS NULL;
```

Consequências

Relatórios de vendas inconsistentes, erros de *NullPointer* na aplicação e perda da integridade lógica do negócio.

Problema 2: Inconsistência em Exclusões (Integridade)

Ao excluir um registro pai sem proteção, os filhos perdem a referência e tornam-se inválidos.



Clientes(id, nome) **Pedidos**(id, id_cliente, valor)

id_cliente referencia Clientes

Exemplo SQL (Sem Constraints)

```
DELETE FROM Clientes WHERE id = 1;
-- Agora a consulta abaixo retorna dados sem sentido:
SELECT * FROM Pedidos WHERE id_cliente = 1;
```

Problema 3: Inconsistência em Atualizações (Integridade)

Alterar a Chave Primária de um pai invalida instantaneamente todos os filhos.



Produtos(<u>id</u>, nome) **Estoque**(<u>id_prod</u>, qtd)

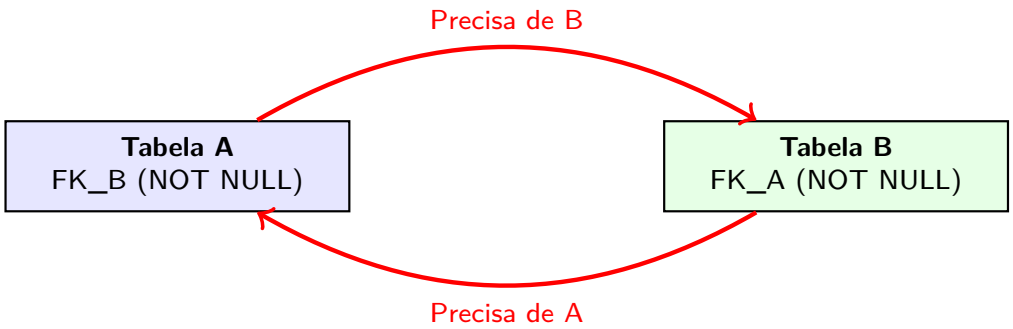
id_prod referencia Produtos

Exemplo SQL (Sem ON UPDATE CASCADE)

```
UPDATE Produtos SET id = 'P99' WHERE id = 'P1';
-- Os registros de estoque continuam apontando para 'P1',
-- que não existe mais na tabela de produtos.
```

Problema 4: Dependências Circulares (Integridade)

Ocorre quando duas tabelas dependem uma da outra para existir.



Tabela_A(id, fk_b) **Tabela_B**(id, fk_a)

fk_b referencia B *fk_a referencia A*

O Impasse do SQL

```
INSERT INTO A (id, fk_b) VALUES (1, 10);
-- Erro: B 10 não existe!
INSERT INTO B (id, fk_a) VALUES (10, 1);
-- Erro: A 1 não existe!
```

Solução 1: Permitindo Nulos e Atualização Tardia

A maneira mais simples de resolver o impasse é permitir que uma das chaves estrangeiras aceite valores **NULL** temporariamente, quebrando a obrigatoriedade inicial.

Passo a Passo da Inserção

```sql
-- 1. Inserir na Tabela A deixando a FK nula (fk_b deve ser NULL)
INSERT INTO A (id, fk_b) VALUES (1, NULL);

-- 2. Inserir na Tabela B referenciando o ID de A
INSERT INTO B (id, fk_a) VALUES (10, 1);

-- 3. Atualizar a Tabela A com o ID de B recém-criado
UPDATE A SET fk_b = 10 WHERE id = 1;
```

Vantagem

É a solução mais compatível entre diferentes SGDBs e fácil de entender logicamente.

Solução 2: Restrições Postergáveis (DEFERRABLE)

Alguns bancos (PostgreSQL, Oracle, SQLite) permitem configurar a restrição para ser verificada apenas no **COMMIT** da transação.

Configuração e Uso (Exemplo Conceitual)

```
-- Definir a FK como postergável na criação da tabela
CREATE TABLE A (
  id INT PRIMARY KEY,
  fk_b INT REFERENCES B(id) DEFERRABLE INITIALLY DEFERRED
);

-- Realizar as inserções dentro de uma transação
BEGIN TRANSACTION;
  INSERT INTO A (id, fk_b) VALUES (1, 10);
  INSERT INTO B (id, fk_a) VALUES (10, 1);
COMMIT; -- O banco valida tudo somente neste momento!
```

**Nota: No SQLite, use `PRAGMA defer_foreign_keys = ON;` dentro da transação.*

Solução 2: Exemplo Prático no SQLite

No SQLite, podemos adiar a verificação mesmo que a tabela não tenha sido criada com a cláusula `DEFERRABLE`, usando um **PRAGMA** especial.

Uso do PRAGMA defer_foreign_keys

```
-- 1. Garante que as chaves estrangeiras estao ativas
PRAGMA foreign_keys = ON;

BEGIN TRANSACTION;
  -- 2. Forca o adiamento das verificacoes nesta transacao
  PRAGMA defer_foreign_keys = ON;

  INSERT INTO Tabela_A (id, fk_b) VALUES (1, 10); -- Sem erro!
  INSERT INTO Tabela_B (id, fk_a) VALUES (10, 1);
COMMIT; -- A integridade e validada aqui.
```

Dica

O `PRAGMA defer_foreign_keys` é redefinido automaticamente para `OFF` assim que a transação termina (`COMMIT` ou `ROLLBACK`).

Problema 5: Performance e Índices (Integridade)

Verificar a integridade em tabelas gigantescas sem índices causa lentidão extrema.



Busca Sequencial Lenta...

Solução SQL

```
-- Sempre indexar as chaves estrangeiras:
CREATE INDEX idx_pedidos_cliente ON Pedidos(id_cliente);
```

Pedidos(id, id_cliente, valor) *id_cliente referencia Clientes*

- **Dica**: A maioria dos bancos não cria índices automáticos para FKs!

Ações Referenciais: ON DELETE e ON UPDATE (Teoria)

O que são?

São gatilhos automáticos definidos na criação da Tabela que determinam como o banco de dados deve reagir quando um registro referenciado (Pai) sofre uma alteração ou exclusão.

- **ON DELETE**: Define o comportamento na exclusão do pai.
- **ON UPDATE**: Define o comportamento na alteração da PK do pai.

Ações Possíveis

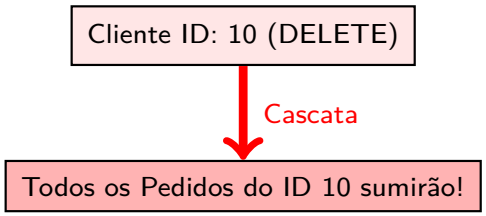
`CASCADE`, `SET NULL`, `SET DEFAULT`, `RESTRICT`, `NO ACTION`

Ações Referenciais: CASCADE (Prática)

A ação **CASCADE** propaga a alteração do pai para todos os filhos vinculados.

Implementação em SQL

```
CREATE TABLE Pedidos (
  id_pedido INT PRIMARY KEY,
  id_cliente INT,
  FOREIGN KEY (id_cliente) REFERENCES Clientes(id)
  ON DELETE CASCADE
  ON UPDATE CASCADE
);
```



Clientes(id, nome) **Pedidos**(id, id_cliente, valor)

id_cliente referencia Clientes

Ação SET NULL: Preservando Registros Filhos

Exemplo de Implementação: SET NULL

```
CREATE TABLE Produtos (
  id_produto INT PRIMARY KEY,
  id_categoria INT NULL,
  FOREIGN KEY (id_categoria) REFERENCES Categorias(id)
  ON DELETE SET NULL
);
```



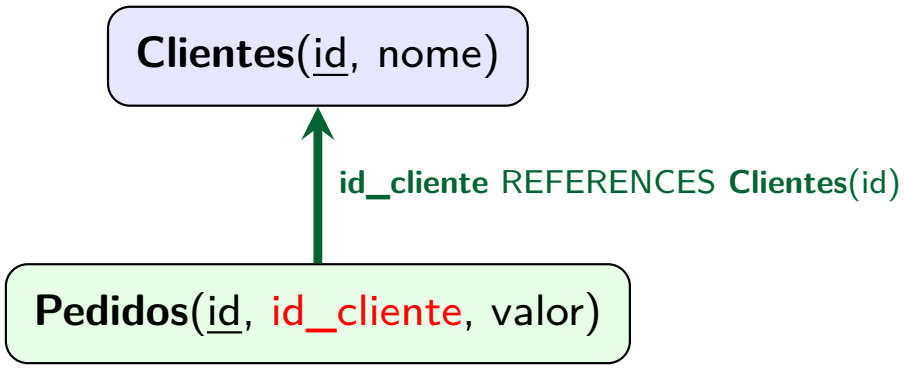
Categorias(id, nome) **Produtos**(id, id_cat, nome)

id_cat referencia Categorias

Exercício 1: Integridade em Vendas (Desafio)

Enunciado

Crie a estrutura de tabelas para um sistema de vendas. Ao excluir um **Cliente**, todos os seus **Pedidos** devem ser removidos automaticamente para evitar registros órfãos.



Clientes (id, nome)
Pedidos (id, id_cliente, valor)
id_cliente referencia Clientes

Resolução do Exercício 1

Solução SQL

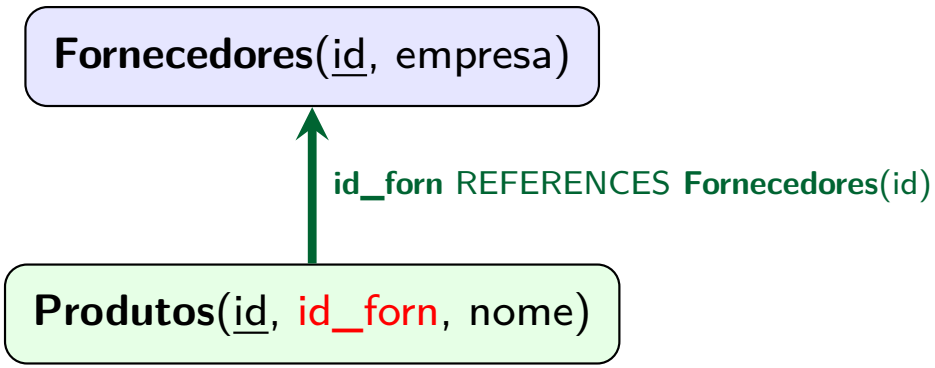
```sql
-- Tabela de Clientes
CREATE TABLE Clientes (
    id INT PRIMARY KEY,
    nome VARCHAR(100)
);

-- Tabela de Pedidos com CASCADE
CREATE TABLE Pedidos (
    id INT PRIMARY KEY,
    id_cliente INT,
    valor DECIMAL(10,2),
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id)
    ON DELETE CASCADE
);
```

Exercício 2: Gestão de Estoque (Desafio)

Enunciado

Modele as tabelas **Fornecedores** e **Produtos**. Se um fornecedor for removido do sistema, os produtos vinculados a ele **não** devem ser excluídos, mas devem ficar sem fornecedor definido.



Fornecedores (id, empresa)
Produtos (id, id_forn, nome)
id_forn referencia Fornecedores

Resolução do Exercício 2

Solução SQL

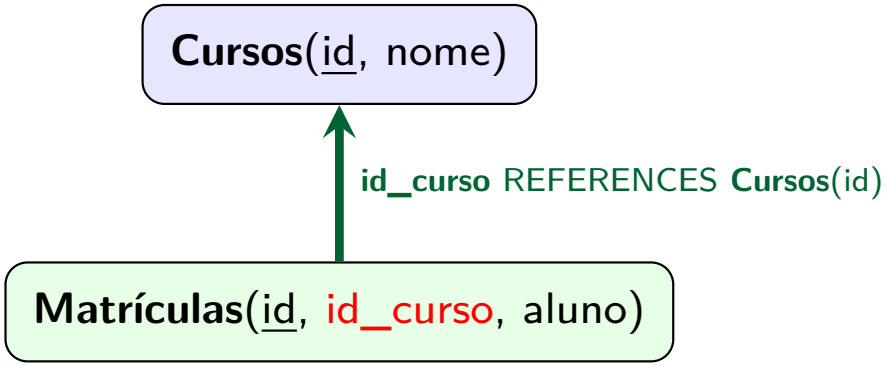
```sql
-- Tabela de Fornecedores
CREATE TABLE Fornecedores (
    id INT PRIMARY KEY,
    empresa VARCHAR(100)
);

-- Tabela de Produtos com SET NULL
CREATE TABLE Produtos (
    id INT PRIMARY KEY,
    id_forn INT NULL, -- Deve permitir nulo!
    nome VARCHAR(100),
    FOREIGN KEY (id_forn) REFERENCES Fornecedores(id)
    ON DELETE SET NULL
);
```

Exercício 3: Restrição de Segurança (Desafio)

Enunciado

Crie a relação entre **Cursos** e **Matrículas**. O sistema deve **impedir** a exclusão de um curso caso existam alunos matriculados nele.



Cursos (id, nome)
Matrículas (id, id_curso, aluno)
id_curso referencia Cursos

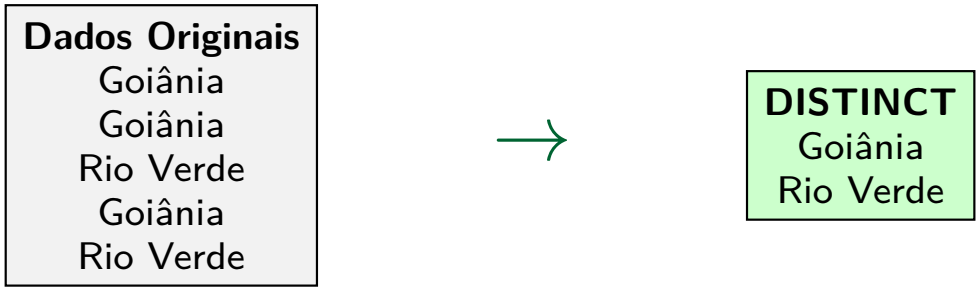
Resolução do Exercício 3

Solução SQL

```sql
-- Tabela de Cursos
CREATE TABLE Cursos (
    id INT PRIMARY KEY,
    nome VARCHAR(100)
);

-- Tabela de Matriculas com RESTRICT
CREATE TABLE Matriculas (
    id INT PRIMARY KEY,
    id_curso INT,
    aluno VARCHAR(100),
    FOREIGN KEY (id_curso) REFERENCES Cursos(id)
    ON DELETE RESTRICT -- Ou NO ACTION
);
```

DISTINCT: Filtrando Valores Únicos



Exemplos SQL

```sql
-- Lista de cidades onde temos clientes
SELECT DISTINCT cidade FROM clientes;

-- Combinacao unica de cidade e estado
SELECT DISTINCT cidade, estado FROM clientes;
```

ORDER BY: Ordenando a Bagunça

Sem Ordem

B	50
A	10
C	30



ORDER BY Letra ASC

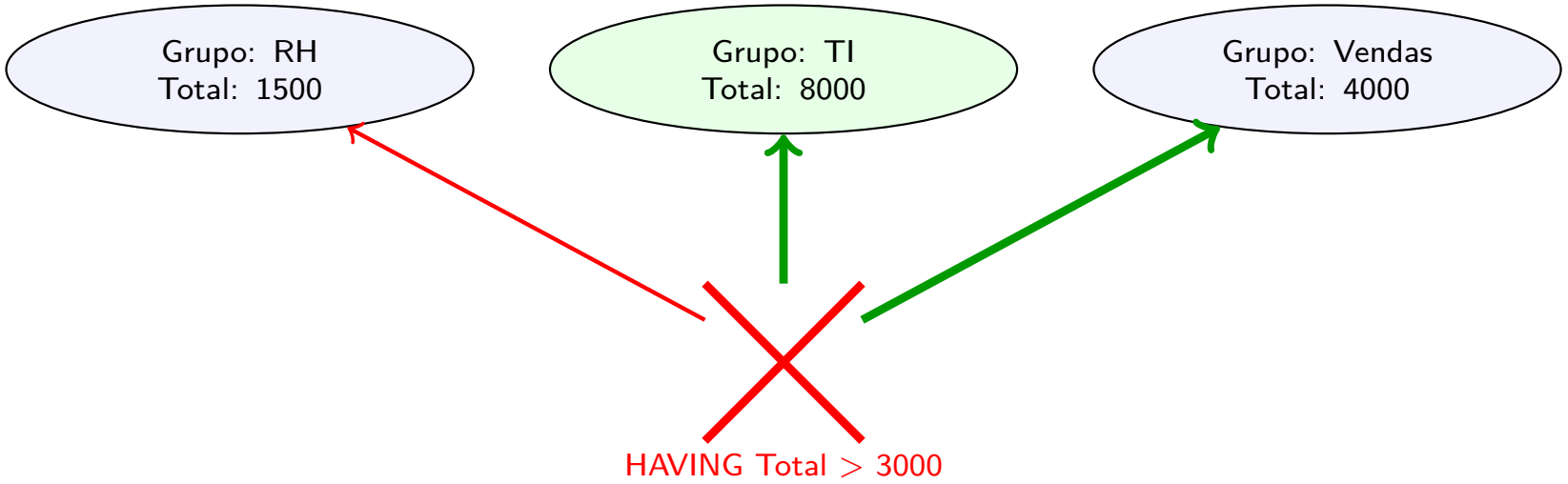
A	10
B	50
C	30

Exemplos SQL

```sql
-- Ordenar por salario do maior para o menor
SELECT nome, salario FROM funcionarios ORDER BY salario DESC;

-- Ordenar por departamento (A-Z) e depois por nome
SELECT * FROM funcionarios ORDER BY depto ASC, nome ASC;
```

HAVING: O "WHERE" dos Grupos



Exemplo SQL Real

```sql
SELECT depto, SUM(vendas) AS total
FROM vendedores
GROUP BY depto
HAVING SUM(vendas) > 100000;
```

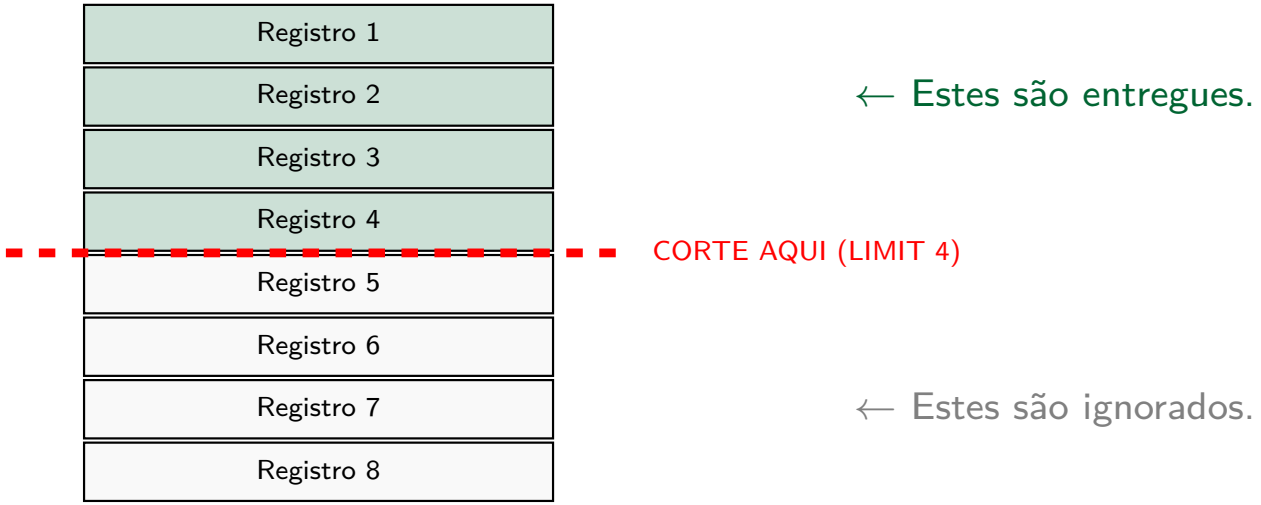
LIMIT: Definindo a Quantidade (Teoria)

O Sinal de Parada

O **LIMIT** diz ao banco de dados: "Não importa quantos registros você encontre, pare de me entregar dados assim que atingir o número X".

- **Analogie**: É como pedir apenas as 3 primeiras fatias de uma pizza, mesmo que ela tenha 8.
- **Controle**: Essencial para criar listas de "Top 10" ou destaques na tela inicial de um site.
- **Eficiência**: Economiza memória do servidor e banda de rede.

LIMIT: O "Corte" nos Dados (Prática)



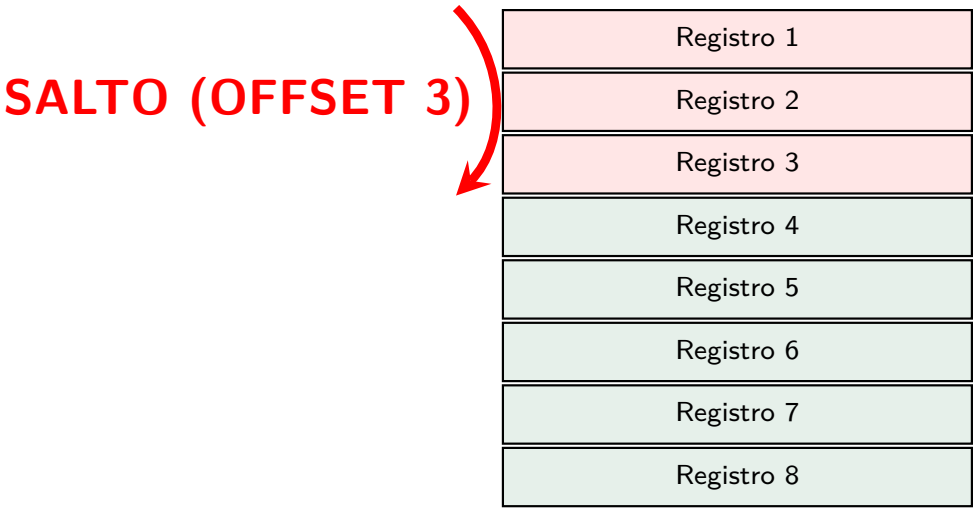
OFFSET: Definindo o Ponto de Partida (Teoria)

O Sinal de Salto

O **OFFSET** diz ao banco de dados: "Ignore os primeiros X registros que você encontrar e comece a me entregar a partir do próximo".

- **Analogie**: É como começar a ler um livro a partir da página 50, ignorando as 49 anteriores.
- **Importância**: Sem o OFFSET, você estaria sempre preso na primeira página de resultados.
- **Regra**: O OFFSET sempre "pula" os dados antes de o LIMIT "cortar" os dados.

OFFSET: O "Salto" sobre os Dados (Prática)



← Ignorados (Pulados).

← Entrega começa aqui.

A Matemática da Paginação (Teoria)

Como calcular as páginas?

Para implementar uma paginação (como no Google ou em uma loja virtual), usamos uma fórmula simples para calcular os valores de cada cláusula:

$$\textbf{LIMIT} = \text{Itens por página}$$
$$\textbf{OFFSET} = (\text{Página Desejada - 1}) \times \text{Itens por página}$$

- **Página 1**: $(1-1) \times 10 = \textbf{OFFSET 0}$
- **Página 2**: $(2-1) \times 10 = \textbf{OFFSET 10}$
- **Página 3**: $(3-1) \times 10 = \textbf{OFFSET 20}$

Paginação na Vida Real (SQL)

Imagine um sistema com 10 produtos por página:

Consultas para cada Página

```sql
-- Pagina 1 (Produtos 1 ao 10)
SELECT * FROM produtos LIMIT 10 OFFSET 0;

-- Pagina 2 (Produtos 11 ao 20)
SELECT * FROM produtos LIMIT 10 OFFSET 10;

-- Pagina 3 (Produtos 21 ao 30)
SELECT * FROM produtos LIMIT 10 OFFSET 20;
```

Aviso Importante

Sempre use **ORDER BY** com paginação! Sem ordem, o banco pode pular registros diferentes a cada consulta, bagunçando a lista.

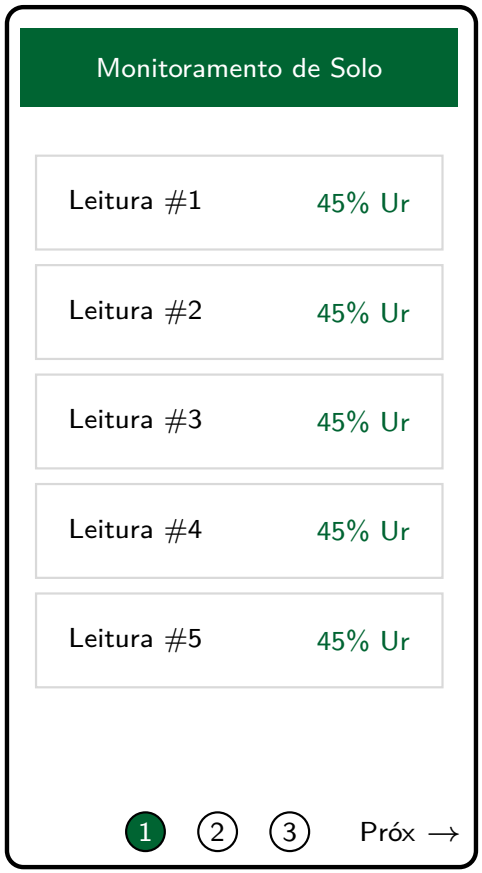
Contexto: Monitoramento de Solo (IoT)

- Imagine uma fazenda inteligente com sensores de umidade espalhados.
- Cada sensor envia uma leitura a cada **10 segundos**.
- Em um dia, temos **8.640 leituras** por sensor!

O Problema

Como mostrar esses dados em um aplicativo de celular sem travar o dispositivo ou gastar todo o plano de dados do produtor?

Wireframe: Visualizando a Paginação no App



← **LIMIT 5**: Mostra apenas 5 registros.

← **OFFSET**: Calcula qual página exibir.

A Tabela: LeiturasSensor

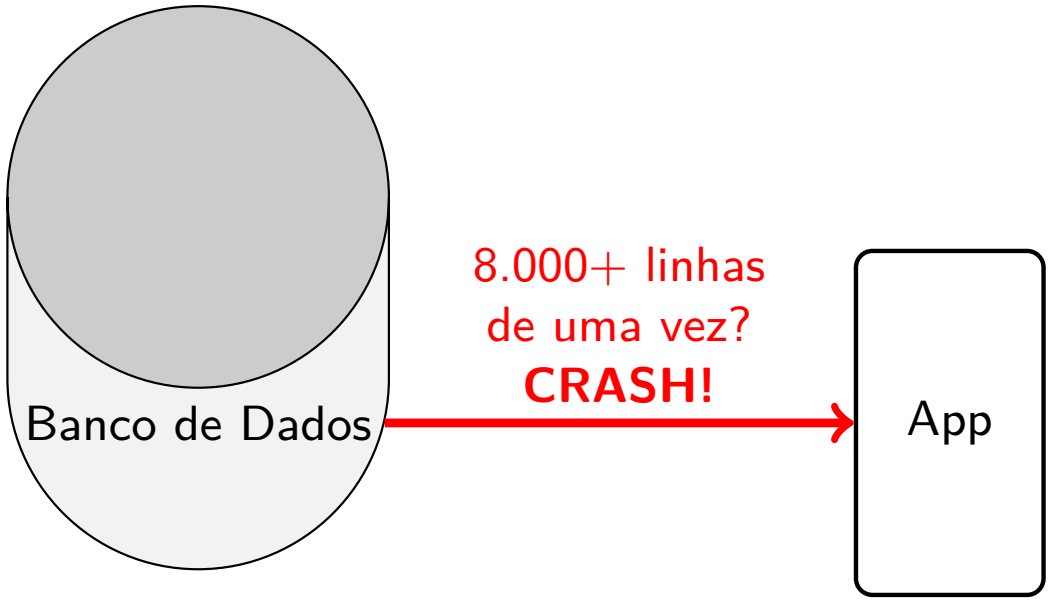
Nossa estrutura de dados para o exemplo:

Schema da Tabela

```
CREATE TABLE LeiturasSensor (
  id SERIAL PRIMARY KEY,
  data_hora TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  umidade DECIMAL(5,2),
  sensor_id INT
);
```

- **Umidade**: Valor em porcentagem (0 a 100).
- **Data_Hora**: Crucial para a ordenação.

O Desafio do Volume



A Solução: Paginação

Dividir as leituras em "pedaços" gerenciáveis (ex: 5 leituras por vez).

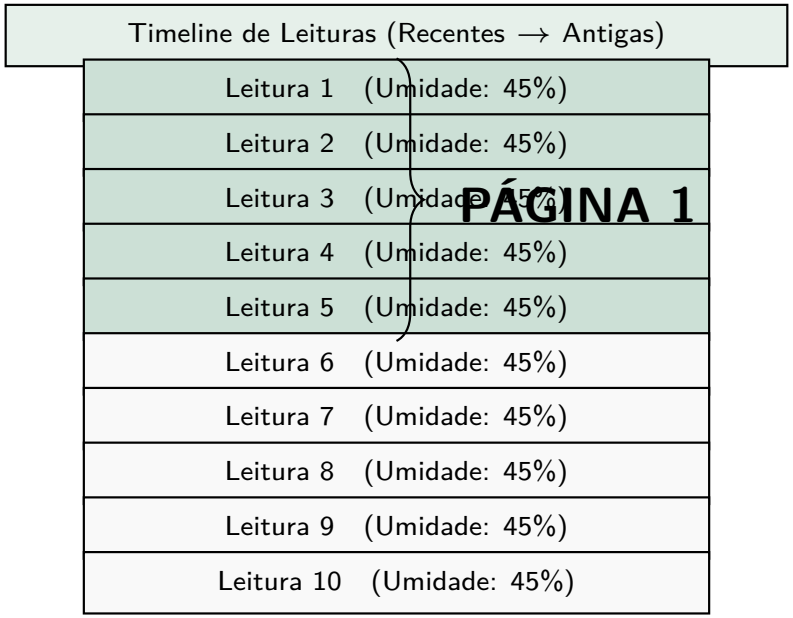
Página 1: As 5 Leituras Mais Recentes

- **Objetivo**: Mostrar o estado atual do solo.
- **Lógica**: Ordenar pelo tempo (descendente) e pegar as primeiras 5.

SQL da Página 1

```
SELECT * FROM LeiturasSensor
ORDER BY data_hora DESC
LIMIT 5 OFFSET 0;
```

Página 1 (Visualização)



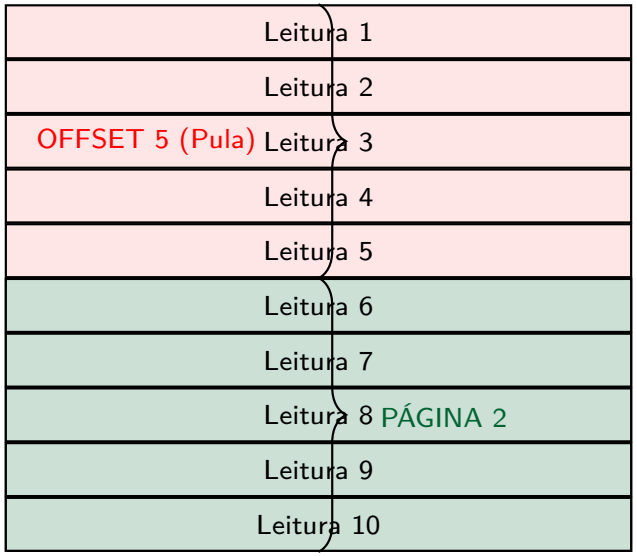
Página 2: Olhando um Pouco para o Passado

- **Objetivo**: Ver o que aconteceu logo antes das últimas 5 leituras.
- **Lógica**: Pular as 5 mais recentes e pegar as próximas 5.

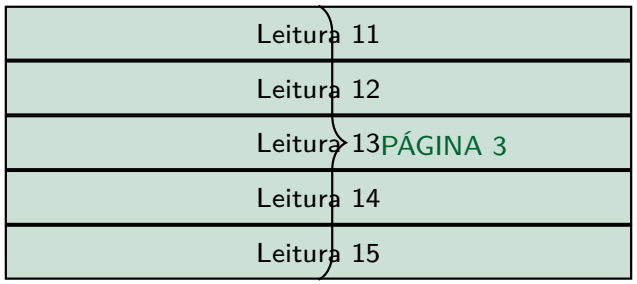
SQL da Página 2

```
SELECT * FROM LeiturasSensor
ORDER BY data_hora DESC
LIMIT 5 OFFSET 5;
```

Página 2 (Visualização)



Página 3 (Visualização)



Página 3: Aprofundando no Histórico

- **Objetivo**: Ver as leituras de alguns minutos atrás.
- **Lógica**: Pular 10 registros e pegar os próximos 5.

SQL da Página 3

```
SELECT * FROM LeiturasSensor
ORDER BY data_hora DESC
LIMIT 5 OFFSET 10;
```

Resumo: Por que ORDER BY é Vital?

Sem Ordem = Caos

Sensores geram dados contínuos. Se você não usar `ORDER BY data_hora DESC`, o banco pode devolver os dados em qualquer ordem:

- Você pode ver uma leitura de ontem na Página 1 e a mesma leitura na Página 2.
- Registros podem "sumir" entre as páginas.
- **Conclusão**: Paginação sem ordenação explícita é um erro grave de desenvolvimento.

Grande Exemplo Final: Unindo Tudo

Consulta Complexa

```sql
SELECT DISTINCT c.cidade, AVG(p.valor) as media
FROM clientes c
JOIN pedidos p ON c.id = p.id_cliente
WHERE p.data > '2023-01-01'
GROUP BY c.cidade
HAVING AVG(p.valor) > 500
ORDER BY media DESC
LIMIT 10 OFFSET 0;
```

- **Explicação**: Busca as cidades com média de pedidos acima de 500 reais, feitas este ano, ordenando pelas cidades que mais gastam, mostrando os top 10.